

H-CSMO: Guarded Multi-Profile Search for Carbon-Efficient and Low-Latency Scheduling in Multi-Region Edge-Cloud Systems

Khalegh Gorji^a, Abbas Mirzaei^{b,*}, Farsad Zamani^a, Ramin Karimi^c, Nasser Mikaeilvand^d

^a*Department of Computer Engineering, South Tehran Branch, Islamic Azad University, Tehran, Iran*

^b*Department of Computer Engineering, Ardabil Branch, Islamic Azad University, Ardabil, Iran*

^c*Department of Computer Engineering, Shahr-e Qods Branch, Islamic Azad University, Shahr-e Qods, Iran*

^d*Department of Computer Science and Mathematics, Central Tehran Branch, Islamic Azad University, Tehran, Iran*

Abstract

Multi-region edge-cloud scheduling couples decisions that are often optimized separately: where a task runs, when it starts, which processor state it uses, and whether migration is worthwhile. Time-varying regional carbon intensity complicates this trade-off because an energy-efficient placement can be carbon-intensive, while consolidation or carbon shifting can extend the critical completion tail. Fixed scalarization is therefore brittle across sparse and SLA-stressed workloads. We present H-CSMO, a guarded multi-profile search framework that jointly controls placement, temporal shifting, five-state DVFS, migration, and sleep/wake behavior. H-CSMO constructs energy-, carbon-, hybrid-, and latency-oriented candidate pools, estimates workload pressure to adapt active-pool size, and allocates the objective-call budget to stochastic refinement around structured schedules. A protected critical-tail stage isolates the latest-finishing tasks and generates targeted turbo or fastest-feasible-node repairs instead of globally accelerating the schedule. The balanced output is selected only from feasible candidates satisfying guards on controllable

*Corresponding author

Email address: A.mirzaei@iau.ac.ir (Abbas Mirzaei)

carbon, controllable energy, gross carbon, gross energy, and makespan relative to reference and activation anchors. An external simulator evaluates every method, while two-layer accounting separates scheduler-controllable effects from the fixed infrastructure floor so that background leakage does not mask scheduling decisions. Experiments cover 24 Google- and Alibaba-derived scenarios: 12 frozen representative and 12 controlled SLA-constrained stress cells, against eight equal-budget methods including NSGA-II and PPO. H-CSMO achieves the best representative rank (1.98/8) and highest normalized carbon–energy–makespan hypervolume in all 24 scenarios. Under SLA stress, it ranks second (2.64/8), keeps the best makespan rank, and significantly outperforms ECO-PSO, TM-MOAOA, and PPO on all five metrics after Holm correction.

Keywords: Carbon-aware scheduling, edge–cloud computing, multi-objective optimization, DVFS, Pareto search, task migration, critical-tail scheduling

1. Introduction

Edge–cloud infrastructures increasingly host services whose computation cannot be treated as a single data-center placement problem. Industrial monitoring, interactive analytics, distributed inference, and scientific services combine short response-time requirements with heterogeneous execution sites, changing network paths, and different regional electricity mixes [1, 2, 3]. A scheduler operating across this continuum must decide not only *where* a task runs, but also when it should start, which performance state the selected processor should use, and whether moving task state across regions is worth the transfer cost. These decisions are coupled: a faster site can shorten a critical completion path while consuming more power; a cleaner region can require a network transfer; and temporal deferral can exploit a lower-carbon interval only when the remaining deadline slack is sufficient.

The carbon dimension makes this coupling especially important. Regional grid intensity can change substantially within the lifetime of a workload, so a placement that is energy-efficient is not necessarily carbon-efficient. Carbon-aware execution studies have shown that spatial and temporal flexibility can reduce emissions, but the attainable gain depends on workload slack, resource heterogeneity, and the carbon variation actually exposed to the scheduler [4, 5, 6]. Recent work has consequently moved from cloud-only energy minimization toward carbon-aware orchestration in the cloud-to-edge

continuum, including function-variant offloading, spatio-temporal federated orchestration, and energy-harvesting edge-cloud allocation [7, 8, 9]. In latency-sensitive systems, however, carbon cannot be optimized independently of deadline feasibility and the power states activated by a placement.

Three technical difficulties dominate this scheduling regime. First, sparse and heavily loaded workloads reward different resource-activation policies. A broad active pool improves headroom under contention but may waste baseline energy when only a few tasks are present. Second, a single scalar objective is brittle when the operating regime changes. Carbon, energy, and completion time can exchange rank across candidate schedules even when all schedules meet their deadlines. Third, migration is neither uniformly harmful nor uniformly beneficial: it can expose a cleaner or faster execution site, but it adds transfer delay, network energy, and transfer-related emissions. These effects become harder to reason about when node sleep/wake transitions and DVFS are available at the same time.

Existing schedulers address parts of this problem through different mechanisms. Carbon-aware methods use forecasts or regional intensity signals to shift computation [10, 6, 7]; Pareto methods preserve several trade-off solutions instead of fixing one weight vector [11, 3]; adaptive metaheuristics alter search behavior as the workload changes [12, 13]; and reinforcement-learning schedulers learn policies from observed system states [14, 15, 16, 17]. These directions are complementary, yet two gaps remain relevant to practical multi-region scheduling. The first is *regime robustness*: a method that is tuned for dense contention can activate too many resources on sparse traces, whereas a conservative energy policy may lengthen the critical tail under SLA pressure. The second is *accounting robustness*: if unavoidable infrastructure leakage is mixed with scheduler-controlled energy, a scheduler can appear insensitive simply because a large fixed floor dominates the reported total.

We address these issues with H-CSMO, a guarded multi-profile search framework for joint placement, temporal shifting, and DVFS control. The scheduler begins from a common dynamic-greedy reference and constructs carbon-, energy-, latency-, and hybrid-oriented candidate pools. Pool size is adapted to the workload pressure estimated from arrival windows and fastest feasible execution times. Candidate schedules are then refined under an exact objective-call budget, so the search effort is directly comparable with competing metaheuristics. A protected critical-tail stage examines the latest-finishing tasks and creates narrowly targeted turbo and fastest-feasible-node repairs. Crucially, the balanced output is not selected by unconstrained

scalarization. It must satisfy fixed guards on controllable carbon, controllable energy, gross energy, gross carbon, and makespan relative to the common reference, the pre-repair anchor, and a deterministic legacy activation anchor.

A second design choice is the evaluation model. We separate the energy and emissions that can change with scheduling decisions from the infrastructure floor that remains even when nodes are in deep-off leakage. Both quantities are reported. The *scheduler-controllable* layer reveals the effect of placement, timing, DVFS, wake transitions, and migration, while the *gross fixed-horizon* layer preserves the complete facility-level result. This distinction avoids treating background leakage as a scheduling gain and makes the observed trade-offs interpretable across workload scales.

The experimental design is deliberately split into two evidence regimes. Twelve frozen representative scenarios retain the original Google- and Alibaba-derived workloads at 30, 60, and 90 tasks. A separate SLA-constrained track contains 12 controlled stress cells created from four 150-task base scenarios and three predeclared workload seeds. Eight methods receive the same external simulator, decision space, feasibility repair, and per-scenario objective-call budget, including NSGA-II and a PPO-based scheduler. Search seeds are summarized inside each scenario before paired statistical inference, so repeated optimizer seeds are not treated as independent workloads.

The main contributions are as follows:

- We formulate a joint edge–cloud scheduling model in which node placement, bounded temporal shifting, five-state DVFS, migration, sleep/wake behavior, and regional carbon intensity are evaluated within one shared decision space.
- We develop a workload-adaptive multi-profile search strategy that constructs energy-, carbon-, hybrid-, and latency-oriented active pools and allocates a fixed evaluation budget across deterministic structure and stochastic refinement.
- We introduce protected critical-tail repair and guard-based balanced selection, which target the latest-finishing tasks without allowing latency repair to ignore predeclared energy, carbon, and gross-accounting limits.
- We introduce transparent two-layer accounting that reports scheduler-controllable energy and emissions separately from the fixed infrastructure floor while retaining gross fixed-horizon values for system-level comparison.
- We provide a reproducible evaluation protocol with eight equal-budget

methods, workload-level paired statistics, Pareto indicators, ablations, sensitivity analysis, exact search-call accounting, and schedule-level artifact replay.

The remainder of the paper is organized as follows. Section 2 reviews carbon-aware, adaptive, learning-based, and multi-objective schedulers. Section 3 defines the system and accounting model. Section 4 presents H-CSMO, followed by its algorithmic properties in Section 5. Section 6 describes the traces, baselines, fairness controls, and statistics. Section 7 reports the results, Section 8 interprets the observed trade-offs, and Section 9 concludes the paper.

2. Related Work

The scheduling problem considered here lies at the intersection of carbon-aware orchestration, multi-objective edge–cloud scheduling, adaptive meta-heuristics, and learning-based control. These areas have developed quickly, but they expose different assumptions about workload slack, adaptation cost, and the way competing objectives are represented.

2.1. Carbon-aware scheduling across the cloud–edge continuum

Carbon-aware execution first gained traction in geographically distributed systems because the same unit of energy can carry different emissions at different places and times. West et al. quantified the practical potential of carbon-aware execution for scientific workflows and showed that the available flexibility, rather than carbon variation alone, determines the achievable benefit [4]. GreenScale by Son et al. extended carbon optimization toward edge computing by considering the interaction between execution location and system carbon cost [5]. Zhai et al. used forecast-driven two-stage scheduling to align geo-distributed workloads with wind availability, emphasizing that temporal prediction can be as important as spatial placement [6].

Recent 2026 work has made the carbon model more tightly coupled to continuum orchestration. Calavaro et al. considered carbon-aware offloading with function variants in serverless cloud-to-edge systems, where application variants change the execution trade-off itself [7]. Al-Shammaa and Al-Hadrawi formulated federated edge–cloud orchestration with dynamic spatio-temporal workload management [8], while Fu et al. studied carbon-aware resource allocation and offloading when edge resources interact with energy harvesting [9]. These studies establish that carbon-aware scheduling is no longer a simple

region-selection problem. The remaining difficulty addressed in this paper is narrower but operationally important: carbon decisions are made jointly with sleep/wake transitions, DVFS, migration energy, and a hard latency tail, and the reported carbon benefit is separated from the non-controllable infrastructure floor.

2.2. Multi-objective and Pareto scheduling

Multi-objective scheduling avoids reducing heterogeneous operational goals to a single metric. Abraham et al. survey cloud task schedulers that use Pareto or weighted multi-objective formulations and identify makespan, energy, utilization, and cost as recurring objectives [11]. In edge-cloud settings, Etheredge et al. proposed PULSE, a distributed scheduler for service-based applications across multi-cluster cloud-edge-IoT infrastructures [3]. Younesi et al. considered multi-objective energy optimization in IoT-integrated cyber-physical systems [18]. These methods motivate preserving trade-off structure rather than presenting a single score as universally optimal.

Several metaheuristic schedulers have also targeted the same metric family. Pournazari et al. proposed energy-centric multi-objective offloading in fog computing [19]; Khaledian et al. combined TOPSIS with multi-objective Archimedes optimization in TM-MOAOA [20]; and Tandon et al. used a hybrid metaheuristic to reduce makespan and imbalance in cloud scheduling [13]. The distinction in H-CSMO is that the Pareto archive is accompanied by named operating profiles and a guarded final selector. The search can therefore retain diverse candidates while the deployed balanced schedule is prevented from purchasing a latency improvement with an unbounded energy or carbon regression.

2.3. Adaptive metaheuristic scheduling

Population-based optimizers remain attractive when the scheduling objective is non-smooth, mixed discrete/continuous, and expensive to differentiate. Satouf et al. studied metaheuristic scheduling for latency-sensitive IoT applications at the edge [12]; Pinky and Verma combined metaheuristics for delay and energy optimization in cloud-fog systems [21]; and Pillai et al. used hybrid metaheuristics for fog-layer resource management [22]. Spider-monkey-based scheduling has also been applied to cloud load balancing and multi-objective task allocation [23, 24, 25].

The common strength of these methods is flexible exploration, but their behavior depends strongly on the candidate representation and on how the

search is seeded. A generic mutation operator can spend much of its budget rediscovering obviously poor schedules when the feasible space includes placement, timing, and DVFS simultaneously. H-CSMO therefore gives structure to the search before stochastic refinement: node pools are ranked by energy, carbon, speed, and hybrid scores; profile-specific constructors build feasible schedules; and stochastic perturbations are concentrated around these structured candidates. This makes the evaluation budget itself part of the algorithmic design rather than an incidental stopping condition.

2.4. Learning-based adaptive scheduling

Deep reinforcement learning provides another route to adaptation by learning a policy over a state/action representation. Shen et al. applied reinforcement learning to task scheduling in heterogeneous end-edge-cloud systems [14]; Anand and Karthikeyan proposed EADRL with adaptive exploration for dynamic edge-cloud scheduling [15]; and Ismail et al. reviewed DQN-, DDQN-, actor-critic-, and related approaches for multi-access edge computing [26]. More recent work focuses directly on multi-objective reward design. RELTO uses PPO with context-aware adaptive reward weighting for reliability-oriented MEC offloading [16], and Sravan and Shaik formulate a DRL scheduler for latency, energy, and SLA optimization in edge-cloud systems [17]. Federated reinforcement learning has also been used to coordinate adaptive stream scheduling across edge and cloud resources [27].

Learning-based schedulers can encode long-range state dependencies that hand-designed search operators do not capture. Their practical cost, however, depends on training, reward calibration, and the stability of the deployment distribution. For this reason, the experiments in this paper include an imitation-warm-started PPO scheduler in the same external decision space and with the same simulator-call budget as the classical optimizers. The comparison is therefore about the quality and cost of decision generation under a shared evaluator, rather than about an artificially advantaged simulator interface.

2.5. Positioning of H-CSMO

Table 1 summarizes the closest methodological directions. The contribution of H-CSMO is not a new carbon signal or a new isolated optimizer operator. It is the combination of (i) joint placement-time-DVFS decisions, (ii) workload-dependent active-pool construction, (iii) explicit multi-profile search, (iv) critical-tail repair, and (v) protected selection under two-layer

Table 1: Positioning of representative scheduling directions relative to the control dimensions used in this study.

Direction	Carbon	Energy	Latency/ SLA	Temporal shift	DVFS	Main distinction relative to H-CSMO
Forecast-driven carbon scheduling [6, 7]	✓	✓	limited	✓	–	Strong spatio-temporal carbon control; does not use the guarded multi-profile critical-tail mechanism studied here.
Distributed multi-context objective scheduling [3]		✓	✓	context	–	Distributed Pareto/service placement with a different decision representation and power-state model.
Adaptive DRL [16, 17]	possible	✓	✓	policy dep.	policy dep.	Learns a policy/reward mapping and pays training/inference cost rather than using a bounded profile search.
Metaheuristic hybrids [19, 20, 24]	hy-method	✓	✓	limited	method dep.	Flexible search, typically without the same active-pool, two-layer accounting, and protected tail selector.
H-CSMO	✓	✓	✓	✓	five states	Joint control with equal-budget profile search, two-layer accounting, and protected final selection.

energy/carbon accounting. That combination is designed for the case in which the best carbon schedule, the best energy schedule, and the fastest schedule can be different points in the same scenario.

3. System Model and Problem Formulation

3.1. Infrastructure, workload, and notation

We consider a set of compute nodes $\mathcal{N} = \{1, \dots, M\}$ distributed across six geographical regions. Each node $j \in \mathcal{N}$ has a type $\tau_j \in \{\text{edge, regional, cloud}\}$, region r_j , CPU and memory capacities C_j^{cpu} and C_j^{mem} , and an explicit power-state model. The workload is a set $\mathcal{T} = \{1, \dots, N\}$ of independent tasks. Task i is described by

$$\mathcal{Q}_i = (a_i, d_i, w_i, c_i, m_i, v_i, r_i^0, q_i), \quad (1)$$

where a_i is the arrival time, d_i is the deadline, w_i is observed CPU work, c_i and m_i are requested CPU and memory, v_i is migration state size, r_i^0 is the initial region, and q_i is the service class (urgent, standard, or flexible).

The Google- and Alibaba-derived scenarios preserve source identifiers so that every scheduled row can be replayed against the original task record.

Each task receives one triplet of control variables,

$$\mathbf{y}_i = (j_i, \delta_i, f_i), \quad (2)$$

where j_i is a feasible node, $\delta_i \in [0, 1]$ is a temporal-shift fraction, and $f_i \in \mathcal{F}$ is one of five DVFS states. The complete search vector has dimension $3N$. Internally, node and DVFS choices are represented by normalized keys in $[0, 1)$ so that every compared optimizer acts on the same continuous vector and the same decoder maps that vector to discrete controls.

3.2. Execution time, temporal shifting, and resource feasibility

Let ξ_{τ_j} denote the baseline speed factor of node type τ_j and σ_{f_i} the speed multiplier of DVFS state f_i . The execution time of task i on node j_i is

$$T_i^{\text{exec}} = \frac{w_i}{c_i \xi_{\tau_{j_i}} \sigma_{f_i}}, \quad (3)$$

with $\xi_{\text{edge}} = 0.8$, $\xi_{\text{regional}} = 1.0$, and $\xi_{\text{cloud}} = 1.25$ in the frozen simulator.

When $r_i^0 \neq r_{j_i}$, task state is transferred before execution. Let B_{uv} denote link bandwidth in Gbit/s and L_{uv} the round-trip latency in seconds from region u to v . For migration state v_i expressed in GiB, the simulator transfer time is

$$T_i^{\text{net}} = \frac{8v_i}{B_{r_i^0, r_{j_i}}} + L_{r_i^0, r_{j_i}}, \quad (4)$$

with zero transfer cost for same-region execution. The earliest possible start is

$$e_i = a_i + T_i^{\text{net}}, \quad (5)$$

and the latest target start that leaves one nominal runtime before the deadline is

$$\ell_i = \max\{e_i, d_i - T_i^{\text{exec}}\}. \quad (6)$$

The temporal key selects

$$\hat{s}_i = e_i + \delta_i(\ell_i - e_i). \quad (7)$$

The actual start t_i^s is the earliest time no earlier than $\hat{s}_i$ for which concurrent reservations satisfy

$$\sum_{k: t \in [t_k^s, t_k^f)} c_k \leq C_{j_i}^{\text{cpu}}, \quad (8)$$

$$\sum_{k: t \in [t_k^s, t_k^f)} m_k \leq C_{j_i}^{\text{mem}}, \quad (9)$$

where $t_i^f = t_i^s + T_i^{\text{exec}}$. If the decoded candidate misses d_i , the common evaluator first retries the same node at the performance DVFS state with no temporal shift; if the deadline is still missed, it evaluates all task-feasible nodes under the same repair setting and chooses the earliest feasible completion. This repair is external to every optimizer and therefore does not provide a method-specific advantage.

The workload makespan and deadline statistics are

$$T_{\max} = \max_{i \in \mathcal{T}} t_i^f - \min_{i \in \mathcal{T}} a_i, \quad (10)$$

$$\text{DMR} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[t_i^f > d_i], \quad (11)$$

$$\text{Tard} = \sum_{i=1}^N \max(0, t_i^f - d_i). \quad (12)$$

3.3. DVFS, sleep/wake, and facility power

The five DVFS states are $\mathcal{F} = \{\text{eco-low, eco, balanced, performance, turbo}\}$. State f scales nominal idle and dynamic powers by α_f^{idle} and α_f^{dyn} . Let $\mathcal{A}_j(t)$ be the tasks executing on node j at time t . When $\mathcal{A}_j(t) \neq \emptyset$, the simulator uses

$$P_j^{\text{act}}(t) = \text{PUE}_{r_j} \left[P_j^{\text{idle}} \max_{i \in \mathcal{A}_j(t)} \alpha_{f_i}^{\text{idle}} + P_j^{\text{dyn}} \sum_{i \in \mathcal{A}_j(t)} \frac{c_i}{C_j^{\text{cpu}}} \alpha_{f_i}^{\text{dyn}} \right]. \quad (13)$$

This interval formulation matches the evaluator when tasks with different DVFS states overlap on the same node. Outside active intervals, the node is charged at standby, sleep, or deep-off leakage according to the surrounding reservations and the minimum sleep-time rule. Wake energy is charged whenever a node is first activated or reactivated after a sleep-eligible gap.

Predictive wake is available to every scheduler because arrivals are declared by the scenario; therefore wake latency is planned before a reserved start rather than injected as a method-specific deadline penalty.

The migration network consumes e_{uv} Wh per GiB on an inter-region link. Migration energy and its associated emissions are

$$E_i^{\text{mig}} = v_i e_{r_i^0, r_{j_i}}, \quad (14)$$

$$C_i^{\text{mig}} = \frac{E_i^{\text{mig}}}{1000} \bar{\gamma}_{r_i^0, r_{j_i}}(a_i), \quad (15)$$

where $\bar{\gamma}_{uv}$ is the mean source/destination carbon intensity at the task-arrival time used by the frozen evaluator. Carbon intensity $\gamma_r(t)$ is piecewise constant over 30-minute forecast slots in the experiments.

3.4. Two-layer energy and carbon accounting

A fixed background floor can mask the effect of a scheduler when workloads differ in duration or density. We therefore report both the gross facility result and the portion above an explicitly declared deep-off floor. The deep-off power of node j is

$$P_j^{\text{floor}} = \rho_{\text{off}} P_j^{\text{idle}} \text{PUE}_{r_j}, \quad \rho_{\text{off}} = 0.01. \quad (16)$$

Let t_0 be the scenario origin and $t_f = \max_i t_i^f$ the schedule completion time. The finish-bounded gross energy is

$$E_{\text{gross}}^{\text{finish}} = \sum_{j \in \mathcal{N}} \int_{t_0}^{t_f} P_j(t) dt + \sum_i E_i^{\text{mig}}, \quad (17)$$

and the finish-bounded floor is

$$E_{\text{floor}}^{\text{finish}} = \sum_{j \in \mathcal{N}} P_j^{\text{floor}}(t_f - t_0). \quad (18)$$

The scheduler-controllable increment is then

$$E_{\text{ctrl}} = \max(0, E_{\text{gross}}^{\text{finish}} - E_{\text{floor}}^{\text{finish}}). \quad (19)$$

For gross comparison, the evaluator extends each schedule to the common horizon

$$H = \max\{t_f, \max_i d_i, \max_i a_i\}, \quad (20)$$

which is identical across feasible schedules for the same scenario. The fixed-horizon gross energy is

$$E_{\text{gross}} = E_{\text{gross}}^{\text{finish}} + \sum_j P_j^{\text{floor}} \max(0, H - t_f). \quad (21)$$

Emissions are integrated over the same power intervals using the region-specific carbon process. Denoting this integration operator by $\Gamma[\cdot]$, we report

$$C_{\text{ctrl}} = \max(0, C_{\text{gross}}^{\text{finish}} - C_{\text{floor}}^{\text{finish}}), \quad (22)$$

$$C_{\text{gross}} = C_{\text{gross}}^{\text{finish}} + C_{\text{floor}}^{\text{post}}. \quad (23)$$

The gross quantities are never relabeled as controllable savings; both layers appear in the results.

3.5. Optimization target

The scheduling problem is not reduced to one universal scalar. Its primary objective vector is

$$\mathbf{g}(\mathbf{y}) = (C_{\text{ctrl}}, E_{\text{ctrl}}, C_{\text{gross}}, E_{\text{gross}}, T_{\text{max}}, V_{\text{mig}}), \quad (24)$$

subject to the resource constraints in Eq. (9) and deadline feasibility in Eq. (12). $V_{\text{mig}} = \sum_i v_i \mathbb{I}[r_i^0 \neq r_{j_i}]$ is reported together with migration energy and emissions. The algorithm maintains several profile-specific scalar utilities only to guide search and final operating-point selection; Pareto dominance is computed directly from the physical metrics in Eq. (24).

4. H-CSMO: Guarded Multi-Profile Search

4.1. Design overview

H-CSMO operates on the shared $3N$ -dimensional representation in Eq. (2). Its search is organized around four operating profiles rather than one fixed scalarization: balanced, carbon, energy, and latency-guarded profiles. Each profile changes three things together: the node-ranking criterion used to construct active pools, the admissible DVFS/time-shift options, and the utility used to rank candidate schedules. The balanced profile is the reported default operating point; the other profiles populate the archive and expose alternative carbon, energy, and latency trade-offs.

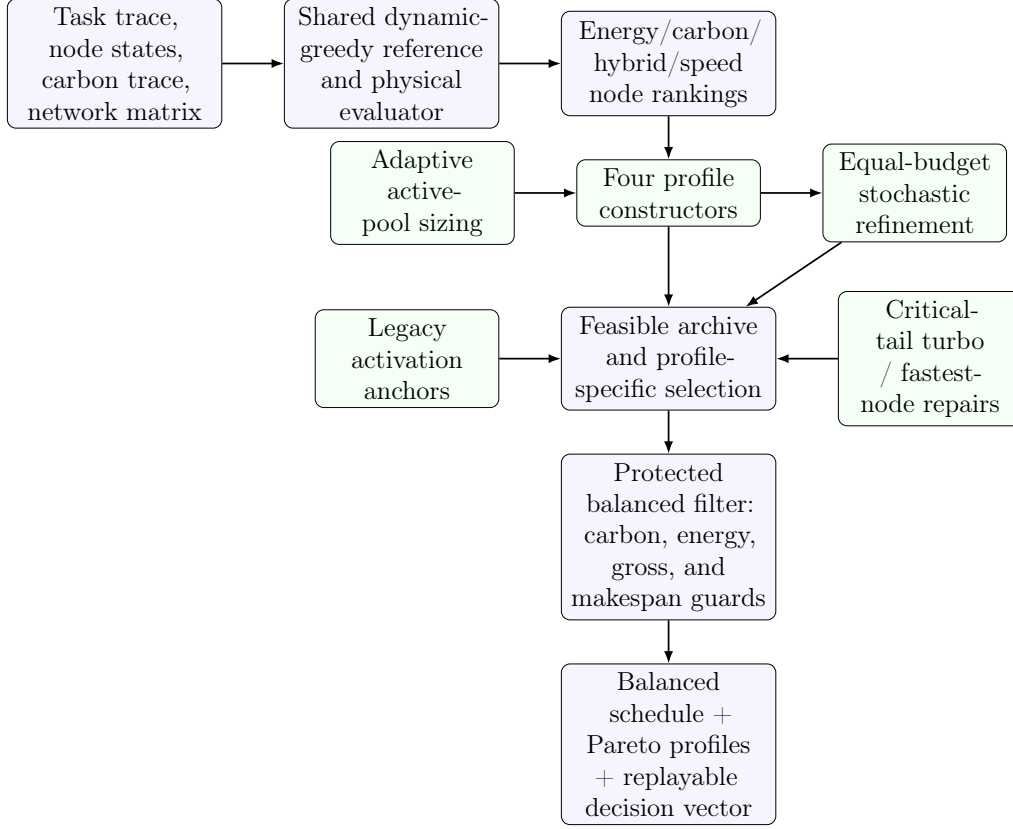


Figure 1: Execution flow of the frozen H-CSMO scheduler used in the reported experiments. All candidate schedules are evaluated by the same external simulator used for the baselines.

Figure 1 shows the control flow. The shared simulator first evaluates a dynamic-greedy anchor. Structured constructors then generate profile-specific candidates from ranked node pools. A workload-pressure estimator changes the number of active nodes used by the representative-regime constructors. The remaining objective-call budget is spent on grouped stochastic perturbations around structured leaders and on diversification around the common anchor. The final phase creates targeted repairs for the tasks that finish last, filters all feasible candidates through fixed protection rules, and returns the fastest protected balanced candidate. The non-dominated records remain available for Pareto analysis.

4.2. Profile-aware construction

For each task, a constructor considers a ranked subset of feasible nodes and a profile-specific set of DVFS/time-shift actions. Candidate nodes are ranked by static estimates of energy, carbon, speed, or a hybrid score. Within a selected pool, the constructor enumerates feasible actions and computes four local attributes: incremental energy, incremental carbon, normalized finish time, and whether a new node must be activated. Each attribute is min-max normalized over the current task's options.

Let κ_i denote deadline criticality,

$$\kappa_i = \min \left(1, \frac{T_i^{\text{fast}}}{\max(d_i - a_i, T_i^{\text{fast}}, \epsilon)} \right), \quad (25)$$

where T_i^{fast} is the fastest runtime among the enumerated options. Each profile starts from a base weight vector $\mathbf{b}^{(p)} = (b_E, b_C, b_T, b_A)$. The finish-time weight increases with criticality,

$$\omega_T^{(p,i)} = \min \left(0.88, b_T^{(p)} + 0.40\kappa_i \right), \quad (26)$$

and the remaining weight is redistributed proportionally over energy, carbon, and activation:

$$(\omega_E, \omega_C, \omega_A) = \frac{1 - \omega_T}{b_E + b_C + b_A} (b_E, b_C, b_A). \quad (27)$$

The local option score is

$$S_{i,o}^{(p)} = \omega_E \hat{E}_{i,o} + \omega_C \hat{C}_{i,o} + \omega_T \hat{T}_{i,o} + \omega_A \hat{A}_{i,o}, \quad (28)$$

and the lowest-scoring feasible option is reserved before the next task is processed. Tasks are considered by arrival time, then deadline, then descending work, which preserves the time structure of the trace while giving earlier deadlines precedence within an arrival batch.

The allowed temporal-shift actions also depend on service class. Urgent tasks are never deliberately delayed. Standard tasks can use performance/balanced execution, while flexible tasks expose larger shift fractions in carbon- and balanced-oriented profiles. This keeps temporal carbon shifting tied to explicit slack instead of applying the same deferral rule to every task.

Table 2: Base profile weights and principal control choices. The finish weight is increased by Eq. (27) for critical tasks.

Profile	b_E	b_C	b_T	b_A	Main DVFS choices
Balanced	0.34	0.34	0.27	0.05	eco, balanced, turbo
Carbon-guarded	0.18	0.57	0.20	0.05	eco, balanced, performance
Energy-guarded	0.58	0.17	0.20	0.05	eco-low, eco, balanced
Latency-guarded	0.12	0.08	0.75	0.05	performance, turbo

4.3. Workload-adaptive active pools

A fixed active-pool size behaves poorly across both sparse and contention-heavy workloads. H-CSMO estimates the required pool size from the maximum demand observed in a sliding 30-minute arrival window. Let T_i^{turbo} be the fastest feasible turbo runtime of task i , and let $\mathcal{W}(t)$ be the tasks arriving in $[t, t + 1800)$. The pressure estimate is

$$D = \max_t \frac{\sum_{i \in \mathcal{W}(t)} T_i^{\text{turbo}}}{1800}. \quad (29)$$

The representative-regime pool size is

$$K = \text{clip} \left(\left\lceil \frac{D}{0.75} \right\rceil, 1, 8 \right), \quad (30)$$

with at least two nodes for the latency profile. Energy-, carbon-, hybrid-, and speed-ranked pools use this value to generate four adaptive constructors. The search additionally evaluates balanced pools of sizes 1, 2, 3, 4, 6 across the four ranking criteria and latency pools of sizes 1–4 across energy, carbon, and hybrid rankings. These deterministic candidates are useful because they directly expose consolidation versus headroom choices before stochastic search begins.

4.4. Equal-budget refinement and Pareto archive

Let B denote the externally assigned objective-call budget. The search begins with the dynamic-greedy anchor, uniform and efficiency-oriented pool vectors, four profile constructors, and criticality repairs. It then alternates between two low-cost perturbation modes until the allocated budget is consumed. Global diversification adds Gaussian noise to the anchor. Grouped mutation selects a contiguous task group and perturbs only node, DVFS, and temporal keys consistent with the active profile; leaders are refreshed

periodically from the current archive. The grouped operator preserves useful constructor structure and avoids repeatedly randomizing all $3N$ coordinates.

Every evaluated vector is stored once using a rounded vector key. Feasible records are compared by the physical objective tuple in Eq. (24); a record is non-dominated if no other feasible record is no worse in all six metrics and strictly better in at least one. The same archive is used for the four profile outputs, which avoids separate optimizer runs for carbon, energy, and latency.

Profile selection uses ratios to the dynamic-greedy reference. For the balanced profile, for example,

$$U_{\text{bal}} = \max(R_C, R_E, R_T) + 0.25(R_C + R_E) + 0.25R_T \quad (31)$$

$$+ 45[R_T - 1.01]_+ + G + 5 \times 10^{-5}V_{\text{mig}}, \quad (32)$$

where $R_C = C_{\text{ctrl}}/C_{\text{ctrl}}^{\text{ref}}$, $R_E = E_{\text{ctrl}}/E_{\text{ctrl}}^{\text{ref}}$, $R_T = T_{\text{max}}/T_{\text{max}}^{\text{ref}}$, and

$$G = 30[R_{E,g} - 1.01]_+ + 30[R_{C,g} - 1.01]_+ \quad (33)$$

penalizes gross-energy and gross-carbon regressions. Carbon-, energy-, and latency-guarded profiles use analogous utilities with profile-specific caps. These utilities guide selection; the Pareto indicators reported later are computed from the unscaled physical metrics.

4.5. Protected critical-tail repair

The final stage targets a failure mode that is easy to miss in energy-oriented schedules: a small number of late tasks can determine makespan even when the rest of the schedule is efficient. Two deterministic activation anchors are first constructed from the cloud node and the edge pool in region 3. The better feasible legacy anchor is retained as a reference. The balanced pre-repair schedule and the legacy anchor are each evaluated with a returned schedule and sorted by decreasing finish time.

For tail sizes $h \in \{1, 2, 4, 8\}$, two repair vectors are generated from each base schedule. The first sets the h latest-finishing tasks to turbo DVFS with no temporal shift. The second additionally moves each selected task to its fastest feasible turbo node. This produces 16 explicit tail candidates. Together with two legacy templates and two schedule-producing tail analyses, the stage reserves exactly 20 objective calls. Consequently, the earlier search receives $B - 20$ calls and the complete method still consumes exactly B calls.

The balanced output is chosen from feasible records satisfying all of the following fixed guards:

$$C_{\text{ctrl}} \leq C_{\text{ctrl}}^{\text{DG}}, \quad (34)$$

$$E_{\text{ctrl}} \leq 1.10E_{\text{ctrl}}^{\text{base}}, \quad (35)$$

$$C_{\text{gross}} \leq C_{\text{gross}}^{\text{DG}}, \quad (36)$$

$$E_{\text{gross}} \leq 1.05E_{\text{gross}}^{\text{base}}, \quad (37)$$

$$T_{\text{max}} \leq 1.01T_{\text{max}}^{\text{DG}}, \quad (38)$$

$$(E_{\text{gross}}, C_{\text{gross}}, T_{\text{max}}) \leq 1.05(E_{\text{gross}}, C_{\text{gross}}, T_{\text{max}})^{\text{legacy}}. \quad (39)$$

Among candidates that pass Eq. (39), selection is lexicographic in makespan, gross energy, gross carbon, and controllable energy. If no repair passes, the pre-repair balanced anchor is retained. The guard therefore cannot force a worse repair merely because a tail candidate is faster.

5. Algorithmic Properties and Computational Cost

The deployed scheduler combines deterministic construction, stochastic search, and an external feasibility-aware simulator. This structure supports several implementation guarantees, but it does not imply global convergence for the mixed discrete/continuous scheduling problem. We therefore state only properties that follow directly from the frozen implementation.

5.1. Feasibility and deadline handling

Resource feasibility is enforced during reservation construction. For every decoded task, the evaluator searches for the earliest start time at which the requested CPU and memory fit the existing reservations on the selected node. If the resulting completion exceeds the deadline, the common repair first removes temporal shifting and raises the selected node to the performance state; if necessary, all feasible nodes are checked under the same repair and the earliest completion is selected. Final profile selection excludes any record with a positive deadline-miss ratio or non-zero total tardiness. Thus, the schedules reported for H-CSMO are feasible by construction under the same repair logic available to every comparator. This is an evaluator property, not a claim that arbitrary instances are always feasible.

Algorithm 1 Guarded multi-profile search in H-CSMO

Require: Shared simulator $\mathcal{S}$, task set $\mathcal{T}$, evaluation budget B , search seed s

Ensure: Balanced schedule, four profile schedules, non-dominated archive

- 1: Evaluate the shared dynamic-greedy anchor and initialize archive $\mathcal{A}$
- 2: Build energy, carbon, hybrid, speed, and workload-adaptive pool candidates
- 3: Evaluate deterministic profile and joint constructors
- 4: Allocate $B - 20$ calls to base search and grouped stochastic refinement
- 5: **while** base budget remains **do**
- 6: Cycle through the four profiles
- 7: Generate anchor diversification or profile-consistent grouped mutation
- 8: Evaluate with $\mathcal{S}$ and update $\mathcal{A}$ and profile leaders
- 9: **end while**
- 10: Evaluate region-3 cloud and edge legacy activation anchors
- 11: **for** base $\in$ {balanced anchor, legacy anchor} **do**
- 12: Sort tasks by decreasing finish time
- 13: **for** $h \in \{1, 2, 4, 8\}$ **do**
- 14: Evaluate turbo/no-shift repair on the h latest tasks
- 15: Evaluate fastest-feasible-node + turbo/no-shift repair
- 16: **end for**
- 17: **end for**
- 18: Form the non-dominated set from feasible records
- 19: Apply Eq. (39) to balanced candidates
- 20: Return the fastest protected candidate and the four profile outputs

5.2. Exact objective-call accounting

Let B be the budget assigned to a method in a scenario. H-CSMO reserves 20 objective calls for two legacy anchors, two schedule-producing tail analyses, and 16 critical-tail candidates, while its earlier search receives $B - 20$ calls. No post-search replay call is counted as a search objective call. The final suite verifies the counter directly for every equal-budget run. As a result, comparisons with ECO-PSO, TM-MOAOA, the adapted metaheuristics, NSGA-II, and PPO use the same number of calls to the external objective evaluator for a given scenario.

5.3. Search cost

Let $C_{\text{eval}}(N, M)$ denote the cost of evaluating one $3N$ -dimensional decision vector on M heterogeneous nodes. The dominant work is reservation placement, migration calculation, and piecewise power/carbon integration. H-CSMO performs exactly B such evaluations, so its dominant cost is

$$\mathcal{O}(B C_{\text{eval}}(N, M)). \quad (40)$$

Candidate construction adds node ranking and local option enumeration. With a bounded active-pool size $K \leq 8$, the constructor overhead is approximately $\mathcal{O}(NK|\mathcal{F}|)$ after node rankings are available. The critical-tail stage evaluates a constant number of candidates because the tail-size set is fixed to $\{1, 2, 4, 8\}$. Archive storage grows linearly with the number of unique evaluated vectors; storing all decision vectors therefore requires $\mathcal{O}(BN)$ values in the reference implementation.

The wall-clock results in Section 7.5 are more informative than asymptotic notation for practical use, because the compared methods differ in Python-level search bookkeeping and PPO additionally incurs neural-network inference and updates. We therefore report both objective-call equality and measured decision time.

5.4. Protected-selection property

The final repair stage cannot replace the pre-repair balanced anchor with a candidate that violates the fixed guard set in Eq. (39). When the admissible set is empty, the base anchor is returned. When it is non-empty, the selected candidate is the fastest member of that set. This yields a simple conditional property: *tail repair can change the deployed balanced schedule only inside the declared carbon, energy, gross-accounting, and makespan protection envelope*. The property is intentionally local to the evaluated candidate set and should not be interpreted as a global optimality theorem.

6. Experimental Methodology

6.1. Trace-derived workloads and evaluation tracks

The workload scenarios are derived from two public production traces: the Google cluster workload released in 2011 and Alibaba Cluster Trace v2017 [28, 29, 30]. We use two scenario families for each dataset. *Trace-dynamic* scenarios retain the trace-derived resource and timing structure, whereas

Table 3: Final experimental design. Search seeds are nested algorithmic repetitions; paired inference is performed after aggregating within scenario/workload cells.

Track	Datasets	Families	Nodes	Tasks	Workload/scenario seeds	Calls
Representative	Google2011v2; Alibaba2017	trace; flexible	30	30	811	156
			30	60	823	216
			30	90	827	276
SLA stress	Google2011v2; Alibaba2017	trace; flexible	50	150	6000, 6101, 6102	396

flexible-dynamic scenarios expose additional scheduling slack while preserving the task resource characteristics. Source task identifiers remain unique in every scenario and are carried into the saved schedule, allowing row-level replay.

The evaluation uses two separate tracks because sparse representative workloads and SLA stress answer different questions. The representative track contains 12 frozen scenarios: two datasets $\times$ two families $\times$ task counts of 30, 60, and 90, all on 30 nodes. Scenario seeds 811, 823, and 827 are tied to the three task counts. These files are byte-for-byte frozen inputs to the final benchmark.

The SLA-constrained track contains 12 controlled stress cells on 50 nodes and 150 tasks. Four base scenarios (two datasets $\times$ two families) are evaluated with workload seeds 6000, 6101, and 6102. Seed 6000 is the original SLA-constrained scenario. Seeds 6101 and 6102 apply service-class-stratified permutations to task arrival times; CPU work, CPU/memory requests, migration state, service class, source ID, and SLA slack are unchanged, and each deadline is reconstructed as the new arrival plus the unchanged slack. We therefore treat these cells as *controlled SLA-constrained workload-seed replications*, not as independent untouched trace replays.

6.2. Carbon, network, and power environment

The environment contains six regions. Carbon intensity comes from a half-hourly six-region NESO forecast trace spanning 1–8 January 2024; the 336 slots per region are replayed cyclically over one week by the simulator. Inter-region placement is charged through the frozen network matrix using round-trip latency, bandwidth, and transfer energy. Same-region execution incurs no migration transfer. Inter-region RTTs range from 8 to 38 ms, bandwidth from 5 to 25 Gbit/s, and transfer energy is 0.06 Wh/GiB. The

Table 4: Representative node-class power parameters used by the shared simulator. CPU/memory capacities are normalized trace-scale capacities.

Type	CPU	Mem.	Idle (W)	Dyn. (W)	Stby. (W)	Sleep (W)	Wake (s)	Wake E. (Wh)
Edge	0.25	0.25	40	60	8	3.2	5	0.174
Regional	0.50	0.50	90	130	18	7.2	15	1.146
Cloud	1.00	1.00	180	240	36	14.4	30	4.375

Table 5: Five-state DVFS control shared by all schedulers.

State	Speed σ_f	Idle mult.	Dynamic mult.
Eco-low	0.70	0.82	0.46
Eco	0.80	0.86	0.55
Balanced	0.90	0.92	0.70
Performance	1.00	1.00	1.00
Turbo	1.10	1.05	1.30

matrix also stores jitter bounds for provenance, but the deterministic evaluator uses the declared base RTT and bandwidth values so that identical decision vectors replay exactly.

Table 4 summarizes the base node classes. Regional PUE ranges from 1.12 to 1.22 across the six regions. The simulator accounts for active dynamic power, standby/sleep states, minimum up/sleep durations, predictive wake latency, and wake energy. The same control model is exposed to every method.

The five-state DVFS multipliers are shown in Table 5. They are applied uniformly across methods. The performance state is the nominal baseline; turbo raises speed by 10% with higher idle/dynamic power, while eco and eco-low trade runtime for lower power.

6.3. Compared methods and fairness controls

The main benchmark contains eight equal-budget methods. ECC-CEE, MEO, and Jellyfish-RTS are adapted implementations: their characteristic search logic is retained, but candidate schedules are represented and evaluated in the shared joint placement–time–DVFS space rather than by private method-specific simulators. This distinction is explicit because the comparison targets scheduling behavior under a common physical evaluator, not line-by-line reproduction of third-party source code.

Table 6: Methods in the main equal-budget benchmark.

Method	Family	Role in the comparison
H-CSMO	Guarded multi-profile search	Proposed joint placement/time/DVFS scheduler with adaptive pools and critical-tail protection.
ECC-CEE-adapted [10]	Carbon-aware edge-cloud	Adapted carbon/latency-oriented search under the common evaluator.
ECO-PSO-Joint [19]	PSO/metaheuristic	Joint-control PSO-style search with equal objective-call budget.
TM-MOAOA-Joint [20]	MCDM + metaheuristic	Joint-control adaptation of TOPSIS/MOAOA scheduling.
MEO-adapted [31]	Multi-objective metaheuristic	Adapted carbon-aware multi-objective search.
Jellyfish-RTS-adapted [32]	Latency-oriented metaheuristic	Adapted latency-sensitive evolutionary search.
NSGA-II [33]	Pareto evolutionary	Four-objective non-dominated sorting with crowding under the common decision vector.
PPO-Joint [34]	Deep RL	PPO policy over joint actions, imitation-warm-started without simulator calls, then trained/evaluated inside the same call budget.

Three search seeds (42, 211, and 307) are used for every stochastic main method. These seeds are algorithmic repetitions, not independent workload samples. For paired statistical inference, metrics are first summarized within each scenario/workload cell; the 12 cells in each track are then the paired units. Every main method receives the same scenario file, node/power model, network and carbon traces, decoder, deadline repair, objective definitions, and objective-call budget shown in Table 3.

For diagnostic context, we additionally retain Dynamic Greedy, the historical H-CSMO-v3C implementation, and a constructor-only scheduler. Appendix controls include *true uniform feasible random* and *baseline-perturbed random (35% mutation)*. The latter is deliberately not described as true random assignment because it mutates a baseline schedule rather than sampling uniformly from the feasible space.

6.4. Metrics and statistical analysis

The primary metrics are scheduler-controllable emissions C_{ctrl} , scheduler-controllable energy E_{ctrl} , gross fixed-horizon emissions C_{gross} , gross fixed-horizon energy E_{gross} , and makespan T_{max} . We also report deadline-miss ratio, total tardiness, migration volume/energy/emissions, active-node count, wall-clock decision time, Pareto coverage, additive epsilon indicator, and normalized three-objective hypervolume. Hypervolume uses controllable carbon, controllable energy, and makespan after common normalization within

each scenario.

The omnibus comparison uses the Friedman test across the eight methods. Pairwise comparisons between H-CSMO and each competitor use a two-sided Wilcoxon signed-rank test on paired scenario values, followed by Holm correction within each metric/track family. Effect size is the matched-pairs rank-biserial correlation computed from positive and negative signed ranks. A 5000-resample scenario bootstrap provides a confidence interval for the median paired difference. These analyses use scenario/workload cells as the statistical unit rather than treating the three search seeds as independent evidence.

6.5. Ablation, sensitivity, and reproducibility

The ablation study includes the full scheduler and equal-budget variants with critical-tail repair removed, adaptive active-pool logic removed, five-state DVFS removed, carbon shifting disabled, migration disabled, race-to-idle disabled, and the legacy protection guard removed. Constructor-only and historical-v3C results are reported as diagnostic references rather than equal-budget ablations. Sensitivity experiments cover evaluation budget, active-pool size, critical-tail size, makespan/energy protection guards, migration-energy threshold, carbon-shift cap, and DVFS policy. Parameters that are not active in the frozen implementation are not presented as tunable factors.

The complete experiment contains 576 main equal-budget runs and 1128 total run/reference/control items. Every completed run is atomically committed with a summary, decision vectors, schedules, evaluation log, per-file SHA-256 manifest, and completion sentinel. The final artifact audit verified all 1128 items. Independent replay of all 576 balanced main profiles reproduced the saved metrics exactly and the numeric schedules to a maximum absolute difference of 5.82×10^{-11} . The software environment used Python 3.12.13 on 64-bit Linux with NumPy 2.0.2, pandas 2.2.2, SciPy 1.16.3, Matplotlib 3.10.0, and CPU-only PyTorch 2.11.0.

7. Results

The results are reported separately for the representative and SLA-constrained tracks. Within each scenario, lower carbon, energy, and makespan are better; method ranks are computed scenario by scenario before averaging. This avoids allowing a single high-magnitude scenario to dominate the comparison.

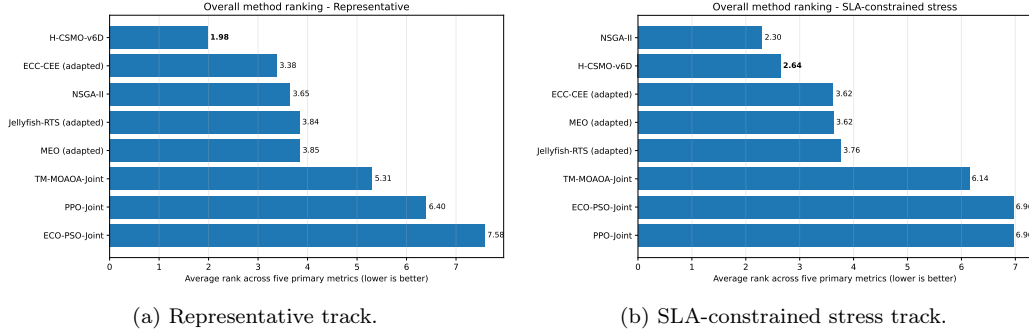


Figure 2: Average rank across the five primary metrics. Lower rank is better. H-CSMO leads the representative track and ranks second under SLA stress.

7.1. Overall ranking on representative workloads

Figure 2a shows the average rank across the five primary metrics on the 12 representative scenarios. H-CSMO obtains an overall average rank of **1.98**, followed by ECC-CEE-adapted (3.38), NSGA-II (3.65), Jellyfish-RTS-adapted (3.84), MEO-adapted (3.85), TM-MOAOA-Joint (5.31), PPO-Joint (6.40), and ECO-PSO-Joint (7.58). The result is not driven by a single objective: H-CSMO has the best average rank for controllable carbon (1.67), controllable energy (1.92), gross carbon (1.67), gross energy (1.92), and makespan (2.75). It places first in 10/12 representative cells for both controllable and gross carbon, 8/12 for both energy measures, and 5/12 for makespan.

Table 7 makes the distinction between the two tracks explicit. On representative workloads, H-CSMO is the strongest method by the aggregate rank used here. Under SLA stress, NSGA-II moves to first overall, while H-CSMO remains second and retains the best average makespan rank.

The representative heatmap in Fig. 3a explains where the rank advantage comes from. Relative to the strongest competing method in each cell, the median H-CSMO improvement is **21.99%** for controllable carbon and 1.95% for controllable energy. Once the fixed infrastructure floor is included, the corresponding median advantages narrow to 0.39% for gross carbon and 0.23% for gross energy, which is expected from Eqs. (19)–(23). Median makespan is essentially tied with the strongest per-cell competitor.

The principal exceptions are confined to the four sparse 30-task cells. In the two Alibaba cells, a specialist competitor becomes the carbon leader; in the two Google cells, an energy-specialized schedule becomes the energy leader. Figure 3a reports the exact cell-level magnitudes rather than averaging

Table 7: Overall average rank across the five primary metrics (lower is better). Bold indicates the best rank in each track.

Method	Representative	SLA stress
H-CSMO	1.98	2.64
ECC-CEE-adapted	3.38	3.62
ECO-PSO-Joint	7.58	6.96
TM-MOAOA-Joint	5.31	6.14
MEO-adapted	3.85	3.63
Jellyfish-RTS-adapted	3.84	3.76
NSGA-II	3.65	2.30
PPO-Joint	6.40	6.96

them away. At 60 and 90 tasks, all eight representative cells remain within 1% of the strongest comparator on every primary metric, while preserving the aggregate rank advantage of H-CSMO. This scale separation is examined further in Section 8.

7.2. SLA-constrained stress performance

The SLA track changes the relative order of the strongest methods. NSGA-II obtains the best overall rank (2.30), followed by H-CSMO (2.64). The proposed scheduler nevertheless remains highly competitive across the five metrics: its average ranks are 2.42 for controllable carbon, 3.00 for controllable energy, 2.42 for gross carbon, 3.00 for gross energy, and **2.38** for makespan. The last value is the best average makespan rank among all eight methods.

Against the strongest competitor selected independently in each SLA cell, H-CSMO has a median controllable-carbon improvement of **12.95%** and a median gross-carbon improvement of **11.24%**. Energy is effectively at parity at the track median: controllable energy is 0.40% higher and gross energy 0.38% higher than the strongest per-cell value. Median makespan difference is zero. This pattern clarifies the role of the profile search under contention: H-CSMO preserves a strong carbon/makespan operating point, while NSGA-II more often finds the lowest-energy point in the same scenario.

The workload-level view remains important. Six of the 12 SLA cells fall within 1% of the strongest comparator on all five primary metrics. In the remaining cells, the competing leader is specialized on one or more individual axes; Fig. 3b retains the exact deviations. The next subsection shows that these single-profile exceptions do not translate into a weaker trade-off set: the

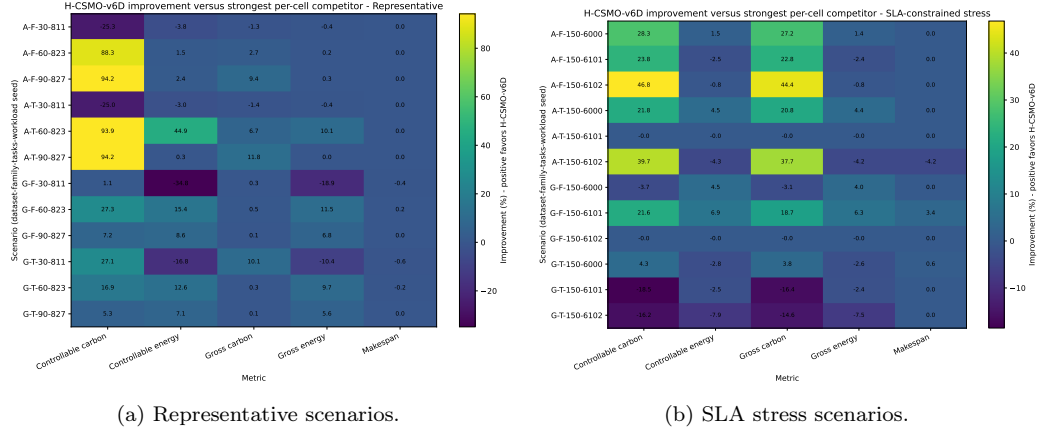


Figure 3: Per-cell improvement of H-CSMO relative to the strongest competitor for each metric. Positive values favor H-CSMO. The full matrix is shown to preserve workload-level variation.

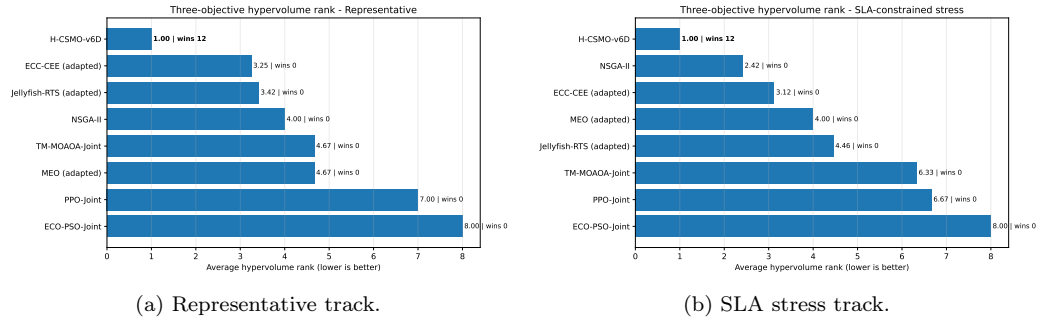


Figure 4: Rank of exact normalized three-objective hypervolume. H-CSMO attains rank 1 in every evaluated scenario.

H-CSMO archive retains the highest normalized three-objective hypervolume throughout the SLA track.

7.3. Pareto quality and hypervolume

The clearest cross-track result is obtained from the Pareto set rather than the single balanced profile. Figure 4 shows the exact normalized three-objective hypervolume ranks using controllable carbon, controllable energy, and makespan. H-CSMO ranks **first in all 24 scenarios**: 12/12 representative cells and 12/12 SLA cells. This remains true in the SLA track even though NSGA-II has the better overall average single-profile rank.

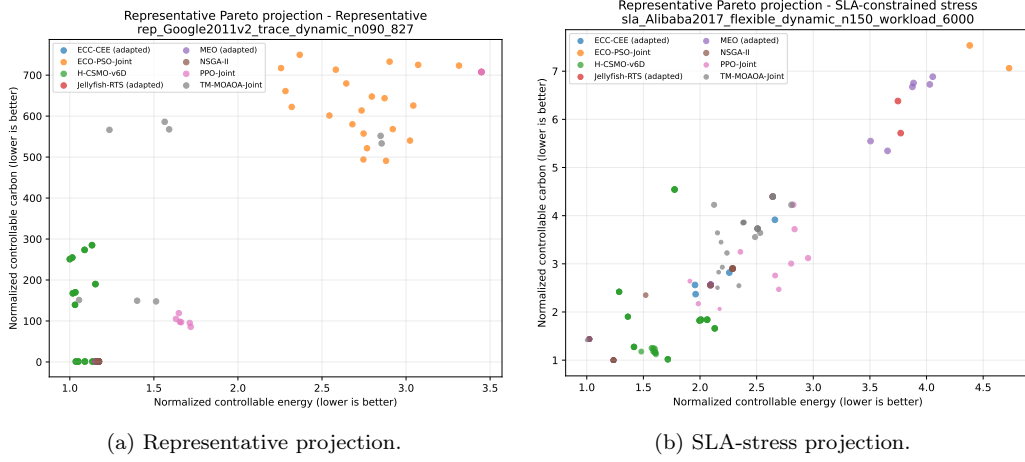


Figure 5: Two-dimensional projection of the three-objective Pareto sets. Marker size encodes the normalized makespan component used in the three-dimensional hypervolume.

Pareto coverage supports the same interpretation with more nuance. On representative workloads, the fraction of competitor front points dominated by the H-CSMO front has mean 0.869 and median 1.0. Under SLA stress, mean coverage decreases to 0.535 and median coverage to 0.420 because the strong NSGA-II/ECC-CEE/MEO/Jellyfish fronts retain non-dominated specialist points. The hypervolume remains higher because H-CSMO covers a broader balanced region of the normalized carbon–energy–makespan space. Example projections are shown in Fig. 5.

7.4. Statistical evidence

The Friedman test rejects equal performance among the eight methods for every primary metric in both tracks ($p \leq 1.12 \times 10^{-7}$ on representative scenarios and $p \leq 8.35 \times 10^{-10}$ on SLA scenarios). Pairwise Wilcoxon–Holm results are summarized in Fig. 6. On representative workloads, H-CSMO has a Holm-significant positive advantage in controllable and gross energy over six of seven competitors. Carbon advantages are Holm-significant over ECO-PSO, TM-MOAOA, and PPO; makespan is significantly better than TM-MOAOA. Against ECC-CEE and several strong Pareto/metaheuristic methods, the direction is often favorable but the 12 paired scenarios do not support a corrected $p < 0.05$ claim for every metric.

Under SLA stress, the result is especially clean against three methods: ECO-PSO, TM-MOAOA, and PPO. H-CSMO is Holm-significantly better

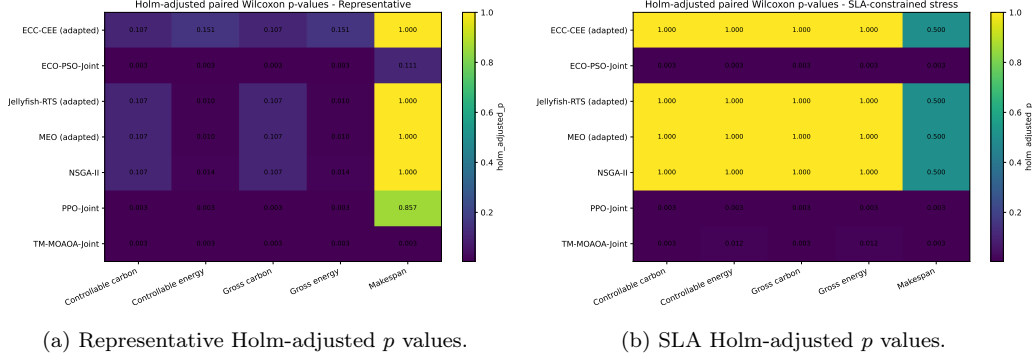


Figure 6: Paired Wilcoxon tests after Holm correction. Scenario/workload cells, not search seeds, form the paired samples.

than each of them for all five primary metrics. The matched-pairs rank-biserial effects are 0.92–1.00 for these comparisons, indicating that the paired direction is consistent across most or all stress cells. Comparisons with NSGA-II, ECC-CEE, MEO, and Jellyfish-RTS are not uniformly significant, which is consistent with their stronger positions in the SLA ranking.

7.5. Runtime and decision overhead

Equal objective-call budgets do not imply equal wall-clock cost because each optimizer performs different search bookkeeping. On representative scenarios, H-CSMO has a median decision time of 9.08 s. The fastest classical competitors are around 5.65–6.14 s, giving a median H-CSMO multiplier of 1.48 and a maximum of 1.68. PPO-Joint is substantially heavier at a 90.56 s median. Thus, H-CSMO is slower than the simplest classical searches but approximately ten times faster than the PPO baseline in this track.

Under SLA stress, the median H-CSMO decision time rises to 50.72 s. ECO-PSO and MEO have medians of 17.59 and 18.30 s; NSGA-II takes 24.56 s; PPO-Joint takes 581.60 s. The median multiplier relative to the fastest classical method is 2.92, with a maximum of 3.17 in the most expensive Google stress cell. Figure 7 shows the complete distributions summarized by method.

7.6. Migration behavior

Migration remains modest on the representative workloads. For H-CSMO, migration energy accounts for a median 0.255% of gross horizon energy and never exceeds 1.137%; migration emissions have a median share of 0.317%

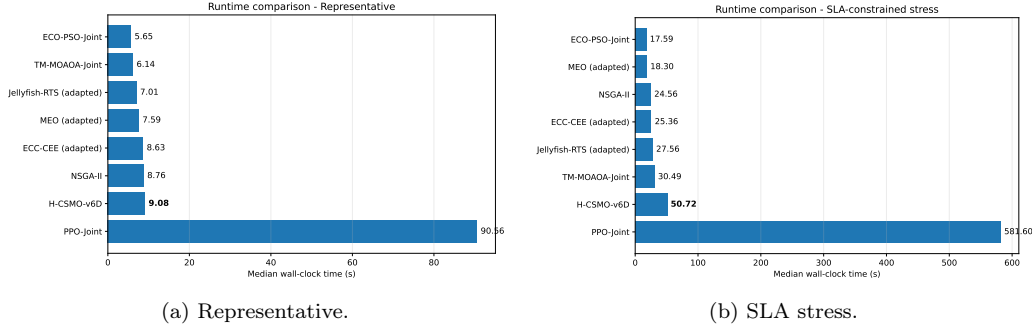


Figure 7: Median wall-clock decision time. The objective-call budget is equal within each scenario; the plot exposes method-specific computational overhead.

and maximum of 0.799%. The SLA track uses migration more aggressively: median energy and emissions shares are 2.20% and 1.95%, while the largest Alibaba stress cases reach 19.97% and 32.35%, respectively. These are the cells in which cross-region movement becomes an explicit part of the carbon/latency trade-off rather than a negligible side effect.

The migration-disabled ablation provides the relevant counterfactual. Across the 12 representative ablation cells, the median full-method improvement over migration-disabled scheduling is 99.77% for controllable carbon, 77.75% for controllable energy, and 59.35% for gross energy, while the median makespan difference is below 0.001%. The high migration fractions observed in a subset of SLA cells are therefore not simply gratuitous movement: the same capability is responsible for large carbon/energy improvements when spatial flexibility matters. Appendix [Appendix B](#) reports the complete ablation plots.

7.7. Ablation and sensitivity

The representative-track ablation identifies migration as the largest measured contributor in the median. Other components are more conditional. Median changes from removing adaptive active-pool control or critical-tail repair are close to zero, and the median selected point is unchanged when carbon shifting or the legacy guard is disabled. This does not imply that those mechanisms are redundant: they are protection and regime-selection operators whose effect is triggered only when the archive contains a candidate that crosses the relevant boundary.

The DVFS ablation exposes a different trade-off. Across the representative ablation cells, removing five-state DVFS has a median 5.97% lower controllable

energy and 2.02% lower controllable carbon than the full method, but a 0.339% higher makespan. In other words, the full controller uses the extra DVFS range to purchase a shorter critical tail at a small energy/carbon cost rather than treating DVFS as an unconditional energy-saving device. Sensitivity plots show the same mechanism: restricted balanced/eco policies can reduce energy, while the full five-state policy reaches the lower makespan operating point. Across all sensitivity settings, deadline-miss ratio remains zero.

8. Discussion

8.1. *Why the Pareto result is stronger than universal single-metric claims*

The full benchmark separates two questions that are often conflated in multi-objective scheduling: whether a method produces a strong *operating point*, and whether it produces a strong *set of alternatives*. On representative workloads, H-CSMO performs well under both views: it has the best overall single-profile rank and the best hypervolume. Under SLA stress, the two views separate. NSGA-II has the better average rank for the selected balanced schedules, largely because it more often reaches the minimum-energy point. H-CSMO nevertheless has the highest three-objective hypervolume in every stress cell and the best average makespan rank. This means that its archive covers a broader useful carbon–energy–latency region even when another method supplies the lowest value of one individual metric.

This distinction matters operationally. A scheduler deployed under a carbon budget, an energy cap, or a latency objective may prefer different points from the same Pareto set. Reporting only the balanced profile would hide that flexibility; reporting only hypervolume would hide the workload-specific regressions of the selected profile. The two views together provide a more faithful description of what the method offers.

8.2. *Sparse-regime behavior and active-pool control*

The four representative cells that are most difficult for H-CSMO are all 30-task scenarios. At that scale, the infrastructure has substantial capacity relative to demand, and a specialist strategy can consolidate aggressively onto one or two highly efficient nodes. H-CSMO deliberately evaluates multiple profile pools and retains headroom for latency and carbon shifts. The additional active capacity can therefore be unnecessary in the sparsest cells, especially when the strongest competitor happens to select an energy-specialized constructor.

The workload-pressure estimator in Eq. (30) reduces this effect by allowing one-node pools, and the protected selector prevents large makespan regressions. The remaining gaps show that sparse-regime scheduling is not simply a smaller version of the contention problem: activation cost dominates sooner than queueing cost. For this reason, the representative results are reported separately by task scale and the heatmap remains part of the main evaluation.

8.3. Interpreting controllable and gross carbon/energy

The two-layer accounting changes how percentage improvements should be read. A 20% change in scheduler-controllable energy can become a sub-percent change in gross fixed-horizon energy when the infrastructure floor dominates the horizon. This is not a contradiction. E_{ctrl} measures the part affected by placement, timing, DVFS, sleep/wake, and migration; E_{gross} answers the facility-level question after restoring the fixed background floor. The same relation holds for carbon.

The representative track illustrates this directly: median controllable-carbon improvement versus the strongest per-cell competitor is 21.99%, while median gross-carbon improvement is 0.39%. Presenting only the controllable metric would overstate the facility-level impact; presenting only the gross metric would make meaningful scheduler differences look negligible. Both layers are therefore necessary for a carbon-aware scheduling study.

8.4. Migration as an active control rather than a free benefit

Migration is beneficial only when the destination advantage exceeds transfer overhead. The representative workloads stay in a regime where that overhead is small. Several Alibaba SLA cells are different: cross-region movement contributes a visibly larger fraction of energy and emissions. The migration-disabled ablation shows why the full method still chooses those moves. Removing migration collapses the scheduler’s spatial flexibility and produces much larger carbon and energy values in the diagnostic case. The practical conclusion is not that migration is cheap, but that it is a controlled resource that should be charged explicitly and used where its benefit is measurable.

8.5. Deployment regime and decision time

The wall-clock results place H-CSMO between classical metaheuristics and the PPO baseline. Its structured constructors, archive handling, and tail analysis are more expensive than ECO-PSO or MEO, but the method

remains roughly an order of magnitude faster than PPO in both tracks. On the representative workloads, median decision time is approximately nine seconds. The SLA track reaches about 51 seconds because each scenario has 150 tasks and a 396-call budget. Such times fit rolling/batch rescheduling more naturally than sub-second per-request dispatch. For tighter online loops, the same architecture can be used with a smaller call budget; the sensitivity experiment shows that reducing the budget to 60–80% preserves the selected point in the tested scenario while lowering runtime.

8.6. Evaluation scope and transferability

The evidence is based on trace-derived simulation with an explicit external power/network/carbon model, not on measurements from a production edge–cloud deployment. The simulator exposes the same controls and repair semantics to every compared method, and all reported schedules were replayed from saved decision vectors, but real platforms can introduce effects not represented here, including interference, uncertain transfer throughput, inaccurate carbon forecasts, and hardware-specific DVFS transition costs. The three adapted baselines are therefore labeled as adapted implementations, and the SLA arrival permutations are described as controlled workload-seed replications rather than untouched trace samples.

These boundaries do not alter the comparative results inside the declared model; they define the next validation step. A deployment study should preserve the same distinction between controllable and gross energy, measure migration traffic directly, and test whether the protected tail selector remains stable when service-time and carbon forecasts are noisy.

9. Conclusion

This paper presented H-CSMO, a guarded multi-profile scheduler for multi-region edge–cloud systems with joint node placement, temporal shifting, five-state DVFS, migration, sleep/wake control, and time-varying regional carbon intensity. The method combines structured profile constructors, workload-adaptive active pools, equal-budget stochastic refinement, and a protected critical-tail stage that can accelerate the latest-finishing tasks only inside fixed carbon, energy, gross-accounting, and makespan guards. A two-layer accounting model separates scheduler-controllable energy and emissions from the fixed infrastructure floor while retaining gross fixed-horizon results.

Across 12 frozen representative scenarios, H-CSMO achieved the best overall average rank among eight equal-budget methods (1.98/8), including the best average rank for all five primary metrics. Its exact normalized carbon-energy-makespan hypervolume ranked first in every representative scenario. Under 12 SLA-constrained workload-seed stress scenarios, NSGA-II obtained the best overall single-profile rank, while H-CSMO ranked second (2.64/8), retained the best average makespan rank, and reduced median controllable carbon by 12.95% relative to the strongest per-cell competitor. The H-CSMO Pareto set achieved the highest normalized three-objective hypervolume in all 24 evaluated scenarios. Paired Wilcoxon tests with Holm correction further showed significant advantages over ECO-PSO, TM-MOAOA, and PPO across all five primary SLA metrics.

The results indicate that the most reliable advantage of H-CSMO is not universal domination of every metric, but the quality of the protected multi-objective operating region it exposes. Future work will transfer the same frozen decision logic to a hardware-backed edge-cloud testbed, incorporate forecast uncertainty directly into candidate scoring, and investigate lower-cost archive updates for sub-second or few-second decision cycles.

Appendix A. Additional Distributions and Convergence

Figures A.8–A.10 report per-scenario normalized distributions for the five primary metrics. These plots complement average ranks by exposing dispersion and outliers. Figure A.11 shows median best balanced utility as the objective-call budget is consumed.

Appendix B. Ablation Study

The equal-budget ablations remove one control from the full method while keeping the shared evaluator and objective-call budget unchanged. Figures B.12 and B.13 report the full-method improvement relative to each ablation. Positive values favor the full scheduler. Constructor-only and historical-v3C entries are diagnostic references and are not equal-budget architectural ablations.

Appendix C. Sensitivity Analysis

Sensitivity is evaluated only for controls active in the frozen implementation. Figure C.14 shows the most consequential families: active-pool size,

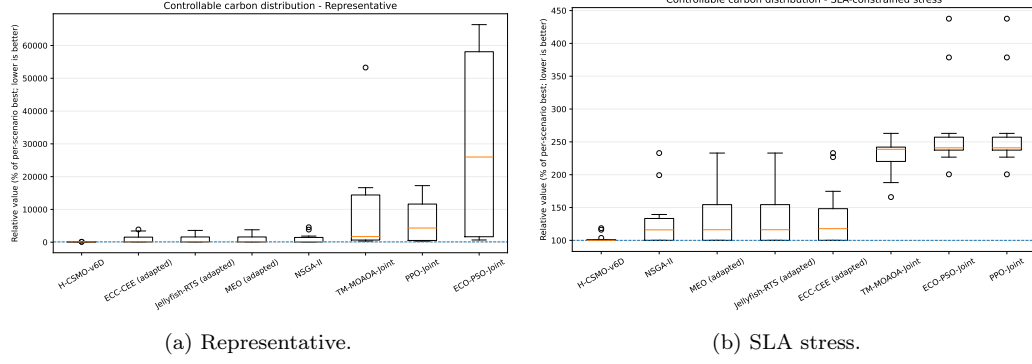


Figure A.8: Controllable-carbon distribution relative to the best method in each scenario.

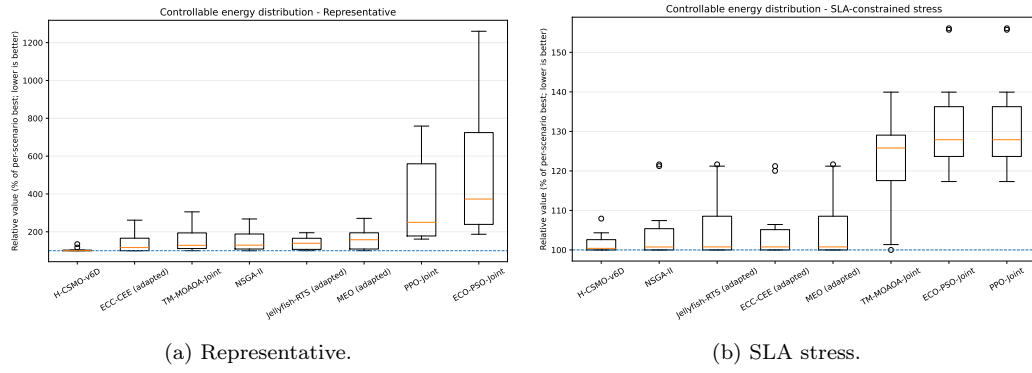


Figure A.9: Controllable-energy distribution relative to the best method in each scenario.

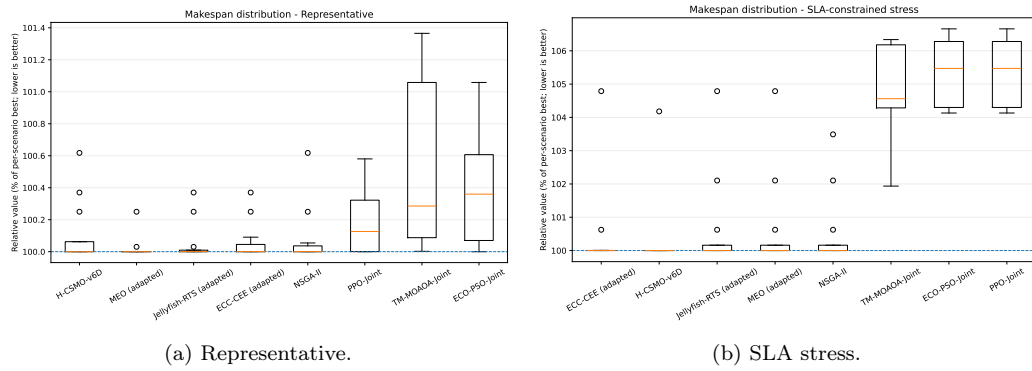


Figure A.10: Makespan distribution relative to the best method in each scenario.

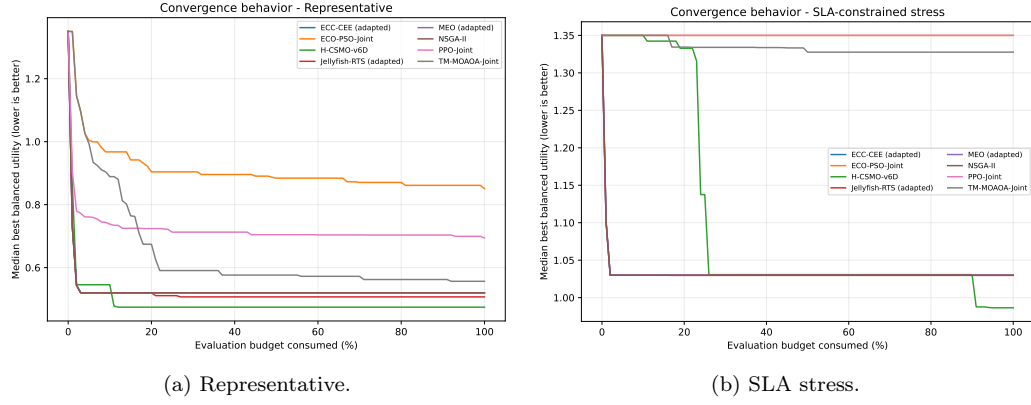


Figure A.11: Median convergence of the best balanced utility as a function of the consumed evaluation budget.

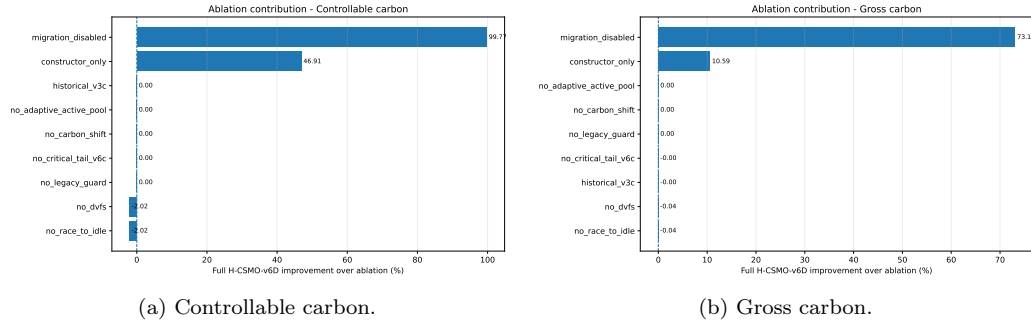


Figure B.12: Ablation results for carbon metrics.

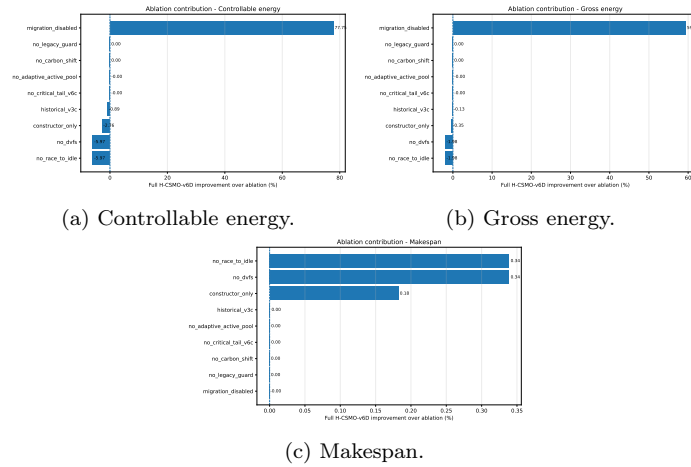


Figure B.13: Ablation results for energy and makespan.

critical-tail size, DVFS policy, and evaluation budget. Guard thresholds and migration/carbon-shift settings are provided in the artifact package and summarized by Fig. C.15. Deadline-miss ratio remains zero for all tested settings.

Appendix D. Migration and Operating Profiles

Figure D.16 relates migration-energy share to gross energy. Figure D.17 shows the median carbon-energy locations of the four operating profiles; marker size reflects the makespan component. These plots make explicit that migration and profile choice are active trade-off controls rather than unreported side effects.

Funding

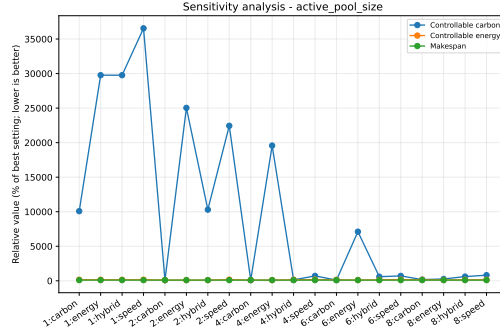
No funding was received for this work.

Data and artifact availability

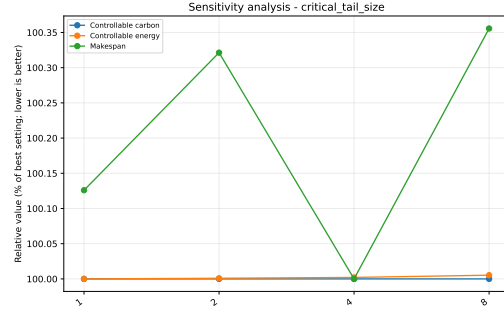
The experiments use trace-derived scenarios from the Google 2011 cluster workload and Alibaba Cluster Trace v2017. The frozen scenario files, run manifests, per-run decision vectors and schedules, aggregate statistical tables, and figure-generation artifacts used for the reported results are available from the corresponding author on reasonable request.

References

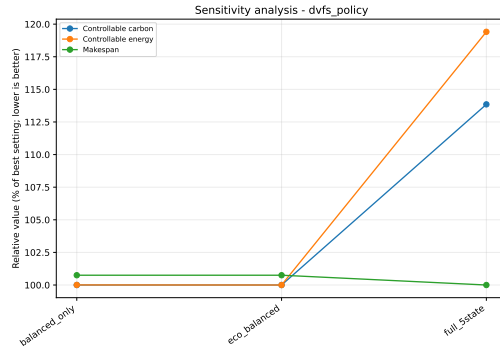
- [1] A. M. Rahmani, A. Haider, P. Khoshvaght, F. Soleimani Gharehchopogh, K. Moghaddasi, S. Rajabi, M. Hosseinzadeh, Optimizing task offloading with metaheuristic algorithms across cloud, fog, and edge computing networks: A comprehensive survey and state-of-the-art schemes, *Sustainable Computing: Informatics and Systems* 45 (2025) 101080.
- [2] J. Pournazari, A. Ullah, A. Al-Dubai, X. Liu, Computation offloading in the edge-to-cloud compute continuum: a survey of federated architectural solutions, *Cluster Computing* 28 (13) (2025) 839.



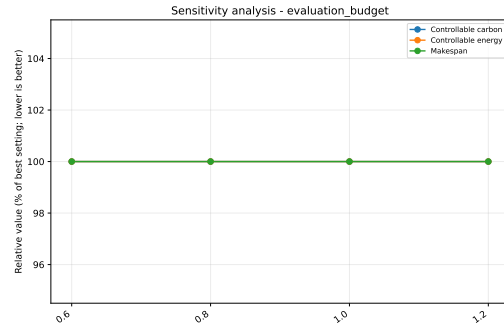
(a) Active-pool size.



(b) Critical-tail size.

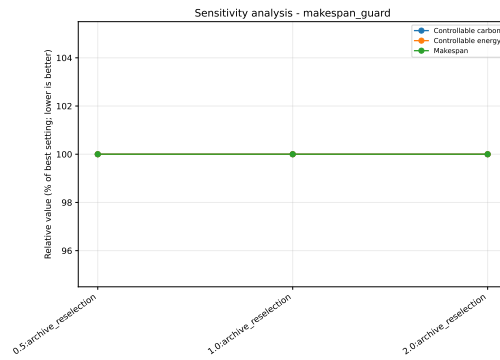


(c) DVFS policy.

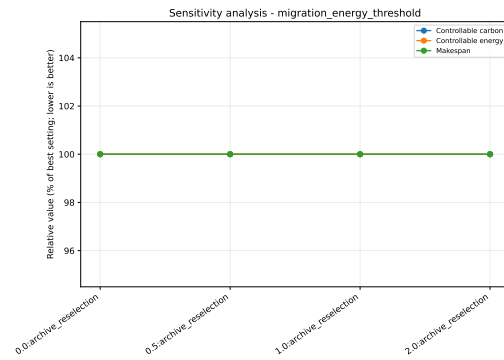


(d) Evaluation budget.

Figure C.14: Sensitivity of active search controls. Lower relative values are better within each metric.



(a) Makespan guard.



(b) Migration-energy threshold.

Figure C.15: Sensitivity of protection controls.

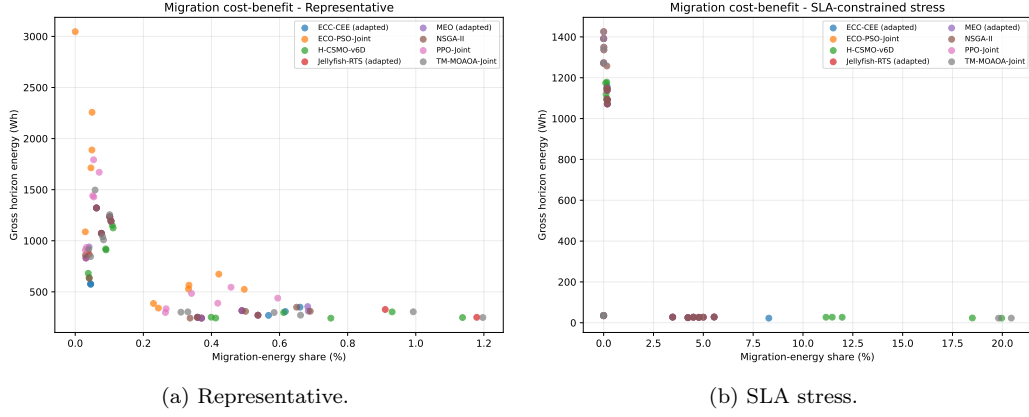


Figure D.16: Migration cost-benefit distributions.

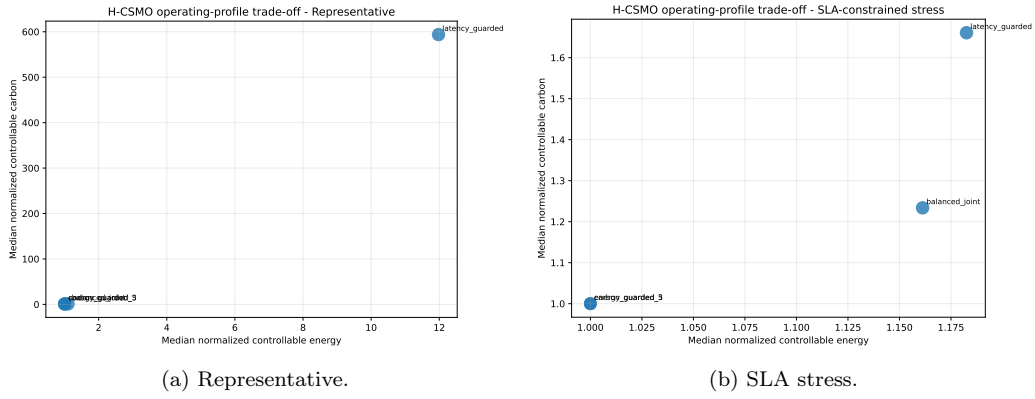


Figure D.17: Median operating-profile trade-offs produced by H-CSMO.

- [3] M. Etheredge, J. A. Poveda, S. Pedratscher, A. Younesi, T. Fahringer, Pulse: Multi-objective scheduling of service-based applications in multi-cluster cloud-edge-iot infrastructures, *Journal of Network and Computer Applications* 248 (2026) 104439. doi:[10.1016/j.jnca.2026.104439](https://doi.org/10.1016/j.jnca.2026.104439).
- [4] K. West, Y. Moawad, F. Lehmann, V. Bountris, U. Leser, Y. Elkhatib, L. Thamsen, [A systematic evaluation of the potential of carbon-aware execution for scientific workflows](#), *Future Generation Computer Systems* 182 (2026) 108453. doi:[10.1016/j.future.2026.108453](https://doi.org/10.1016/j.future.2026.108453). URL <https://doi.org/10.1016/j.future.2026.108453>
- [5] Y. Son, U. Gupta, A. McCrabb, Y. G. Kim, V. Bertacco, D. Brooks, C. J. Wu, Greenscale: Carbon optimization for edge computing, *IEEE Internet of Things Journal* 12 (16) (2025) 32379–32393.
- [6] X. Zhai, G. Zhu, Y. Zhang, X. Guo, Y. Peng, F2s-wss: A forecast-driven two-stage workload scheduling scheme for carbon-aware geo-distributed data centers with wind power integration, *Sustainable Computing: Informatics and Systems* 47 (2025) 101216. doi:[10.1016/j.suscom.2025.101216](https://doi.org/10.1016/j.suscom.2025.101216).
- [7] C. Calavaro, V. Cardellini, F. L. Presti, G. R. Russo, Carbon-aware offloading with function variants for serverless computing in the cloud-to-edge continuum, *Future Generation Computer Systems* 185 (2026) 108670. doi:[10.1016/j.future.2026.108670](https://doi.org/10.1016/j.future.2026.108670).
- [8] A. A. Al-Shammaa, D. H. Al-Hadrawi, Carbon-aware federated orchestration in edge-cloud systems: A multi-objective optimization framework with dynamic spatio-temporal workload management, *Sustainable Computing: Informatics and Systems* 51 (2026) 101398. doi:[10.1016/j.suscom.2026.101398](https://doi.org/10.1016/j.suscom.2026.101398).
- [9] H. Fu, Q. Lu, Y. Yang, Carbon-aware resource allocation and task offloading in EH-assisted edge-cloud systems, *Scientific Reports* (2026). doi:[10.1038/s41598-026-62950-0](https://doi.org/10.1038/s41598-026-62950-0).
- [10] X. Zhai, Y. Peng, X. Guo, Edge-cloud collaboration for low-latency, low-carbon, and cost-efficient operations, *Computers and Electrical Engineering* 120 (2024) 109758.

- [11] O. L. Abraham, M. A. B. Ngadi, J. B. Mohamad Sharif, M. K. Mohd Sidik, Multi-objective optimization techniques in cloud task scheduling: A systematic literature review, *IEEE Access* 13 (2025).
- [12] A. Satouf, A. Hamido
uglu,
Ö. M. G
ül, A. Kuusik, L. Durak Ata, S. Kadry, Metaheuristic-based task scheduling for latency-sensitive iot applications in edge computing, *Cluster Computing* 28 (2025) 143.
- [13] A. Tandon, A. Nayyar, S. Patel, A novel hybrid meta heuristic approach for optimization of makespan and handling imbalance in task scheduling in cloud environments, *Computing* 107 (2025) 170.
- [14] W. Shen, W. Lin, W. Wu, H. Wu, K. Li, Reinforcement learning-based task scheduling for heterogeneous computing in end-edge-cloud environment, *Cluster Computing* 28 (2025) 179.
- [15] J. Anand, B. Karthikeyan, Eadrl: Efficiency-aware adaptive deep reinforcement learning for dynamic task scheduling in edge-cloud environments, *Results in Engineering* (2025) 105890.
- [16] A. Nasir, X. He, T. Wang, H. Shi, Z. Wang, Relto: A reliability-oriented drl approach with context-aware adaptive reward weighting for multi-objective task offloading in mec, *Ad Hoc Networks* 181 (2026) 104065. [doi:10.1016/j.adhoc.2025.104065](https://doi.org/10.1016/j.adhoc.2025.104065).
- [17] P. Sravan, M. A. Shaik, DRL-based multi-objective task scheduling for edge-cloud computing: latency, energy, and SLA optimisation, *Scientific Reports* 16 (2026) 19681. [doi:10.1038/s41598-026-49824-1](https://doi.org/10.1038/s41598-026-49824-1).
- [18] A. Younesi, E. Oustad, M. Abolnejadian, M. Ansari, A. Ejlali, Moticps: Energy optimization on multi-objective task scheduling in iot-integrated cyber-physical systems, *IEEE Transactions on Sustainable Computing* 10 (4) (2025).
- [19] J. Pournazari, A. Ullah, A. Al-Dubai, X. Liu, N. Khaledian, Multi-objective optimisation for energy-centric offloading in fog computing, *Computing* 107 (11) (2025) 1–29.

- [20] N. Khaledian, S. Razzaghzadeh, S. Moazzami, P. N. Kivi, Tm-moaoa: a two-stage task scheduling approach using topsis and multi-objective archimedes optimization in fog-cloud environment, *Computing* 107 (7) (2025) 155.
- [21] Pinky, K. Verma, Enhanced task scheduling with metaheuristics for delay and energy optimization in cloud-fog computing, *Concurrency and Computation: Practice and Experience* 37 (15–17) (2025) e70163.
- [22] S. E. V. S. Pillai, S. Paramasivam, K. Polimetla, S. Dhanasekaran, K. K. Agrawal, S. P. Yadav, J. Logeshwaran, G. Gatto, Optimizing resource management using hybrid metaheuristic algorithm for fog layer design in edge computing, *Systems and Soft Computing* 7 (2025) 200323.
- [23] G. Verma, Load balancing in cloud environment using opposition based spider monkey optimization, *Wireless Personal Communications* 137 (2024) 977–996.
- [24] D. A. Amer, G. Attiya, I. Ziedan, An efficient multi-objective scheduling algorithm based on spider monkey and ant colony optimization in cloud computing, *Cluster Computing* 27 (2024) 1799–1819.
- [25] M. I. Khaleel, The application of hybrid spider monkey optimization and fuzzy self-defense algorithms for multi-objective scientific workflow scheduling in cloud computing, *Internet of Things* 30 (2025) 101517.
- [26] A. A. Ismail, N. E. Khalifa, R. A. El-Khoribi, A survey on resource scheduling approaches in multi-access edge computing environment: a deep reinforcement learning study, *Cluster Computing* 28 (2025) 184.
- [27] S. Lajili, Z. Brahmi, M. N. Omri, R. P. D. Prado, Federated reinforcement learning-based adaptive stream applications scheduling in edge and cloud computing, *Future Generation Computer Systems* 177 (2026) 108235. [doi:10.1016/j.future.2025.108235](https://doi.org/10.1016/j.future.2025.108235).
- [28] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, M. A. Kozuch, Heterogeneity and dynamicity of clouds at scale: Google trace analysis, in: *Proceedings of the Third ACM Symposium on Cloud Computing, SoCC '12*, ACM, 2012, pp. 7:1–7:13. [doi:10.1145/2391229.2391236](https://doi.org/10.1145/2391229.2391236).

- [29] Y. Cheng, Z. Chai, A. Anwar, Characterizing co-located datacenter workloads: An alibaba case study, in: Proceedings of the 9th Asia-Pacific Workshop on Systems, APSys '18, ACM, 2018. doi:[10.1145/3265723.3265742](https://doi.org/10.1145/3265723.3265742).
- [30] D. Shahmirzadi, N. Khaledian, A. M. Rahmani, Analyzing the impact of various parameters on job scheduling in the google cluster dataset, Cluster Computing 27 (6) (2024) 7673–7687.
- [31] J. Pournazari, A. Al-Dubai, X. Liu, R. Akraminejad, A carbon-aware task offloading framework for sustainable fog computing, in: 2025 IEEE International Conference on High Performance Computing and Communications (HPCC), IEEE, 2025.
- [32] V. Nigade, P. Bauszat, H. Bal, L. Wang, Inference serving with end-to-end latency slos over dynamic edge networks, Real-Time Systems 60 (2) (2024) 239–290.
- [33] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Transactions on Evolutionary Computation 6 (2) (2002) 182–197. doi:[10.1109/4235.996017](https://doi.org/10.1109/4235.996017).
- [34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347 (2017).